```
        if @order.save
          Cart.destroy(session[:cart_id])
          session[:cart_id] = nil
»         ChargeOrderJob.perform_later(@order,pay_type_params.to_h)
          format.html { redirect_to store_index_url, notice:
            'Thank you for your order.' }
          format.json { render :show, status: :created,
            location: @order }
        else
          format.html { render :new, status: :unprocessable_entity }
          format.json { render json: @order.errors,
            status: :unprocessable_entity }
        end
      end
    end
```

With this in place, you can now add an item to the cart, check out, and see everything working just as we did before, with the addition of seeing the calls to Pago. If you look at the Rails log when you check out, you should see some logging, like so (formatted to fit the page):

```
[ActiveJob] Enqueued ChargeOrderJob
  (Job ID: 79da671e-865c-4d51-a1ff-400208c6dbd1)
    to Async(default) with arguments:
      #<GlobalID:0x007fa294a43ce0 @uri=#<URI::GID gid://depot/Order/9>>,
      {"routing_number"=>"23412341234", "account_number"=>"345356345"}
[ActiveJob] [ChargeOrderJob] [79da671e-865c-4d51-a1ff-400208c6dbd1]
    Performing ChargeOrderJob
      (Job ID: 79da671e-865c-4d51-a1ff-400208c6dbd1) from
    Async(default) with arguments:
    #<GlobalID:0x007fa294a01570 @uri=#<URI::GID gid://depot/Order/9>>,
    {"routing_number"=>"23412341234", "account_number"=>"345356345"}
[ActiveJob] [ChargeOrderJob] [79da671e-865c-4d51-a1ff-400208c6dbd1]
    Processing check: 23412341234/345356345
```

This shows the guts of how Active Job works and is useful for debugging if things aren’t working right.

Speaking of debugging and possible failures, this interaction really should have a test.

System Testing the Checkout Flow